Dynamic and Iterative Network Flow Analysis

Technical Documentation & Methodology

Duarte Albuquerque • Elijah Tamarchenko • Juan Carlos Meyer • Bruno Croso

Project Overview

This project introduces a novel approach to analyzing football tracking data by combining Graph Theory (Maximum Flow) with advanced, tracking-aware spatial metrics. While traditional football analytics often rely on discrete event data, this model evaluates the continuous, fluid nature of football. By treating the attacking team as a directed capacity network, we quantify the total threat of a possession sequence at any given millisecond and how this develops over time.

The model rests on three interconnected pillars:

- **Spatial Geometry:** Player positions, movement vectors, and field geometry are used to compute threat values with mathematical precision.
- **Network Flow Theory:** The Ford-Fulkerson algorithm treats the attacking team as a directed capacity network, producing a single maximum-threat score per frame.
- **Expected Threat Economics:** Pass edges are weighted by a full utility model: expected gain from a successful pass minus the cost of losing possession on a failed one in a given location.

1. Motivation: Beyond Static Events

Traditional football metrics, such as standard Expected Goals (xG), possess a significant limitation: they are reactive. They evaluate the quality of a chance only at the exact moment a shot is taken. If a player makes a brilliant off-the-ball run that draws defenders away from dangerous zones, or a playmaker executes a line-breaking pass that is intercepted at the last second, traditional event-based xG records nothing.

Our motivation was to capture the latent threat of a team's spatial structure. Football is fundamentally an invasion game played in continuous space. We asked a central question:

> ***"At any given frame of a match, what is the maximum theoretical probability that the current possession will result in a goal, given the positioning of all 22 players? Furthermore, how do specific decisions impact this probability?"***

To answer this, we moved away from isolated event analysis and looked toward electrical engineering and plumbing network mathematics: The Maximum Flow Problem. By treating passing lanes, run corridors, and shot windows as edges with weighted probabilistic capacities, we transform a football pitch into a quantifiable flow network that can be evaluated, compared, and visualized continuously throughout a match.

2. The Core Concept: Football as a Network Flow

In our model, a snapshot of a football match is converted into a directed graph G = (V, E). Every node and edge has a concrete physical interpretation rooted in what is actually happening on the pitch at that millisecond.

2.1 Graph Structure

Carrier (Source)	The identified ball carrier. The single origin of all attacking threat. All flow originates here.
Teammates	Each available teammate acts as a relay node. Flow passes through them via a pass, then continues to goal.
Run Nodes	Virtual nodes representing directional destinations the carrier can drive into — capturing off-ball spatial threat.
Goal (Sink)	The opponent's goal. All attacking flow resolves here. The maximum flow to this node is the frame's threat score.
Pass Edges	Carrier → Teammate. Capacity = pass utility: (xPass × xThreat) − ((1 − xPass) × xCost).
Run Edges	Carrier → Run Node → Goal. Capacity = min(zone control, net xThreat of destination after defensive penalties).
Shot Edge	Carrier → Goal. Capacity = direct xG from the ball's current position. The 'shoot now' option.
xThreat Edges	Teammate → Goal. Capacity = synthetic xG at teammate's position × pass completion probability.

Sample Graphic: Frame-by-Frame Network Flow Structure

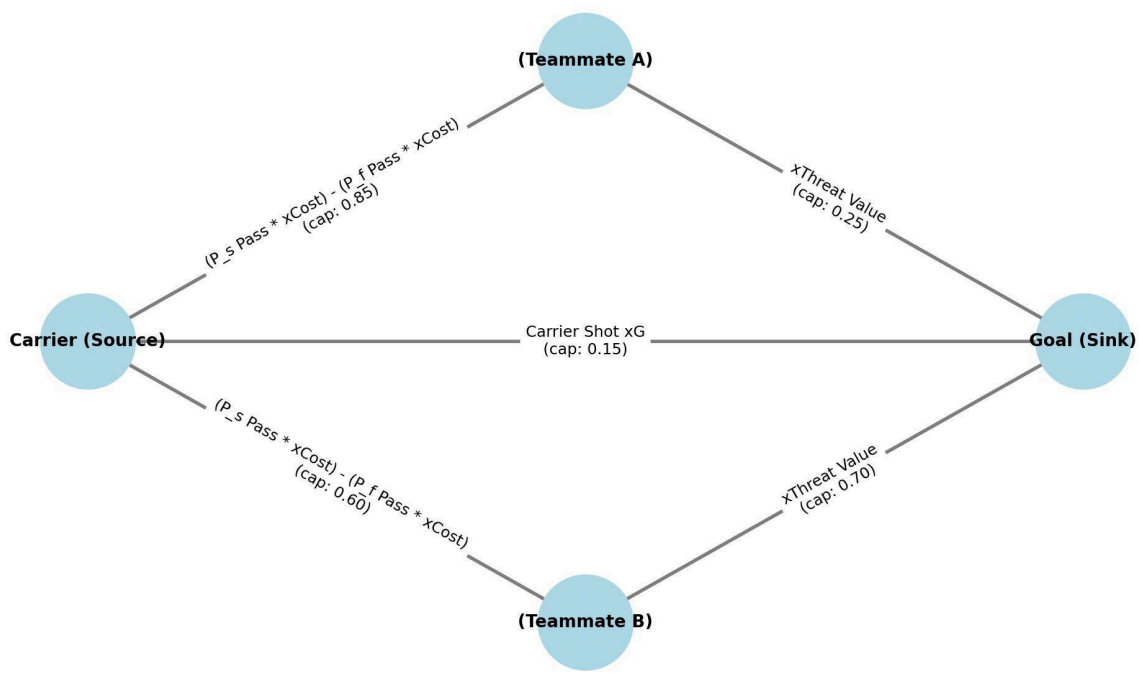


Figure 1: A simplified single-frame network. The carrier distributes threat through pass edges (utility-weighted) and a direct shot edge (xG-weighted). Ford-Fulkerson finds the maximum flow through to Goal.

By applying the Ford-Fulkerson algorithm, we calculate the Maximum Flow from the ball carrier to the Goal. This single number represents the maximum fluid threat of the possession:

- If the ball carrier is isolated with no viable passes, no safe runs, and a poor shooting angle, the flow is low.
- If the team has open passing lanes, dangerous run corridors, and multiple teammates in high-xG positions, the flow spikes.
- Flow is bounded by the bottleneck edge in the network — exactly as in electrical circuits, the weakest link determines maximum throughput.

3. Key Metrics and Their Roles

Tracking-Aware Expected Goals (xG)

Unlike standard xG models that only consider shot location, our xG accounts for the full tracking environment at the moment of evaluation. It layers four compounding factors:

1. **Distance & Angle:** The Euclidean distance to goal and the visible arc between the goalposts form a logistic baseline probability.
2. **Defenders in the Shot Cone:** Each defender physically inside the triangle drawn from shooter to both goalposts applies a compounding 30% penalty.
3. **Defensive Pressure:** Any opponent within 1.5 meters of the ball carrier at shot time applies an additional 40% penalty.
4. **45m Hard Cap:** Any shot from beyond 45 meters is assigned xG = 0.0, preventing unrealistic long-range probabilities.

Pass Utility (Edge Capacity)

> $pass_utility = (xPass \times xThreat) - ((1 - xPass) \times xCost)$

Pass edges are not simply weighted by completion probability. The utility formula rewards high-value targets and explicitly penalizes risky passes by factoring in the xCost — the Expected Threat conceded to the opponent if the pass is intercepted. This gives each edge an economic value: positive utility means the pass is worth attempting; negative means the risk outweighs the reward.

Run Option Modeling

The model evaluates 13 directional run destinations per frame for the ball carrier — 8 forward-facing and 4 backward-facing, plus the carrier's current velocity direction. Each destination is assessed for zone control (can the attacker claim this space before a defender arrives?) and net xThreat (what is the space worth, minus defensive pressure penalties). Only the top-5 positive-value runs enter the network, preventing directional granularity from inflating total flow.

4. What the Flow Score Means

The maximum flow value produced for each frame is a single number that encapsulates the attacking team's total goal-scoring potential at that exact moment. It aggregates three parallel paths of attack:

- **Direct threat:** The xG if the carrier shoots immediately.
- **Pass-and-shoot threat:** The utility of the best available pass multiplied by the receiver's xThreat.
- **Carry threat:** The value of driving into the best available space given current zone control.

Over the course of a match, this value is computed for every frame and plotted as a continuous time series. Spikes indicate dangerous moments even if no shot was taken. Sustained elevation reveals teams that are structurally superior in their attacking phase. Flat or near-zero flow reveals defensive dominance or poor ball-carrier decision contexts.

5. Tactical Application & Performance Analysis

The Maximum Flow metric serves as a high-fidelity diagnostic tool, allowing for a granular evaluation of both collective structures and individual decision-making. By quantifying the "fluidity" of threat, we move beyond assessing what a player did and instead assess the quality of the options they created or denied. Our framework is utilized to evaluate three primary dimensions of the match:

- **Offensive Potential & Decision Trees:** We evaluate how specific off-the-ball runs, line-breaking passes, and body-orientation decisions directly manipulate the system's total flow. This allows us to identify "Flow-Positive" tactical moves—where a player's movement opens multiple high-capacity channels—versus "Flow-Negative" decisions that lead to defensive bottlenecks and system stagnation.
- **Defensive Reliability & Suppression:** Conversely, the model identifies defensive liabilities by tracking which structural lapses allowed the opposition's flow to spike. We quantify defensive efficiency not just by tackles made, but by a defender's ability to "constrict the pipe"—systematically reducing the capacity of the opponent's passing and shooting edges through superior positioning and zone control.
- **Individual Optimization (Action vs. Ideal):** Because every edge in our network is weighted, the model inherently identifies the "Optimal Action" (the path of least resistance to the goal) for every frame. By benchmarking a player's actual decision against this mathematical optimum—and aggregating this delta across all possessions—we can identify the most "calculated" players: those who consistently execute the highest-value actions.

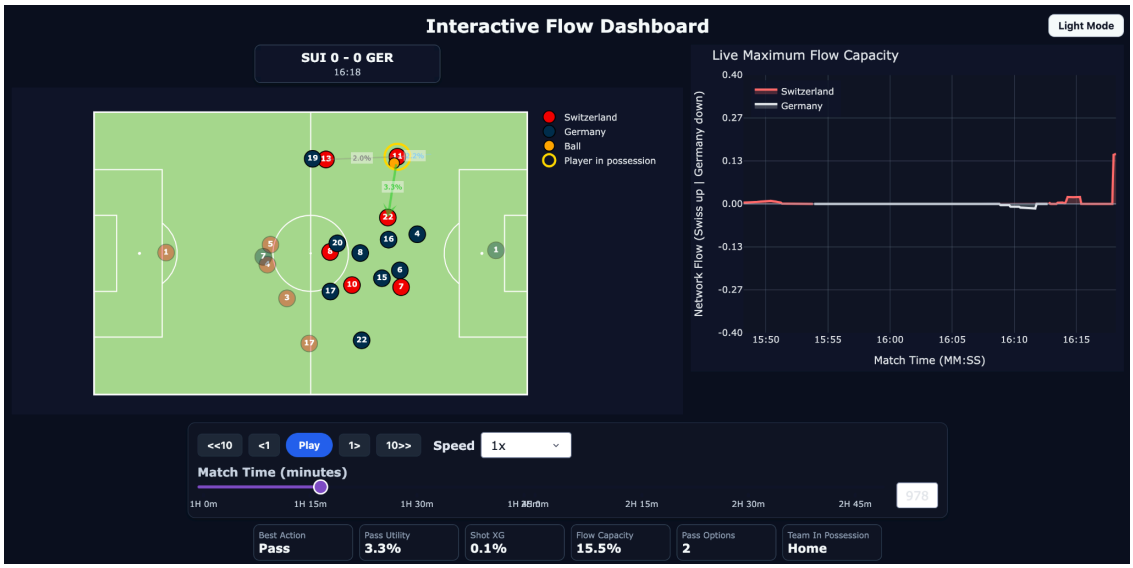
Included in our solution folder is a short clip of the first goal of the game, along with a clip of the flow and passing options during that same play, showing how different players followed through the optimal network flow path to score the goal.

We have also created a dashboard for the entire game, where you can see a 2d model of the game over time, including the different options the player has and a dynamic graph of the flow over the course of the game. To run the dashboard, run

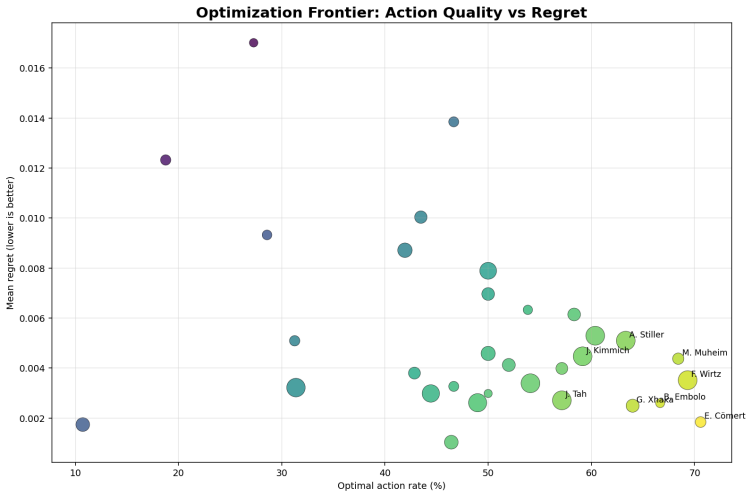
pip install -r requirements.txt

python interactive_dashboard.py

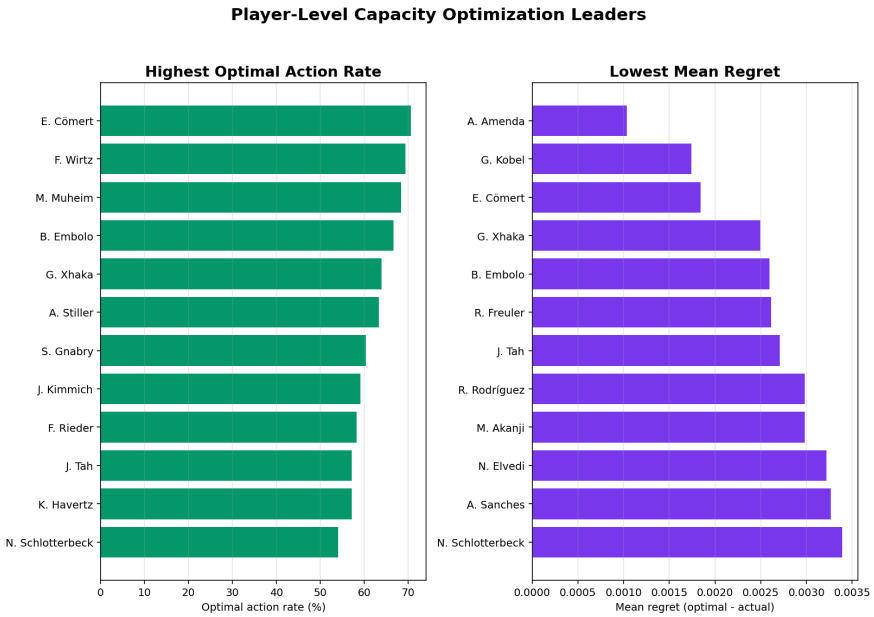
and then open the local Dash URL shown in the terminal (usually 'http://127.0.0.1:8050'). We have included below a screenshot of the dashboard. If the dashboard is not working, the DASHBOARD_GUIDE.md has more information.



We also have some examples of individual player and team analysis derived from the model:

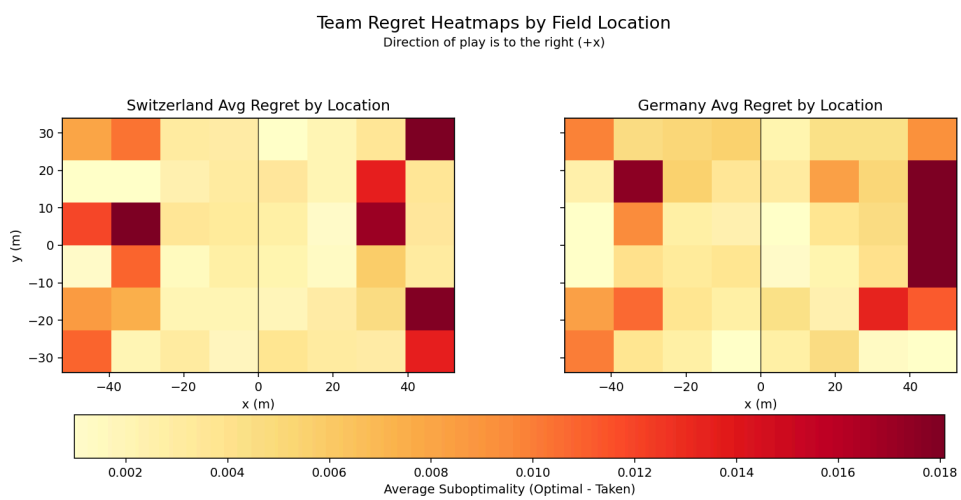


Optimization Frontier: Action Quality vs. Regret This scatter plot visualizes player decision-making efficiency by plotting the percentage of a player's actions deemed mathematically optimal (x-axis) against their mean regret—the average value lost when choosing a suboptimal action (y-axis). The size of the bubble corresponds to the volume of actions the player took. Players positioned in the bottom right quadrant (e.g., E. Cömert, F. Wirtz, and B. Embolo) represent the ideal archetype: they execute a high rate of optimal actions with very little regret.



Player-Level Capacity Optimization Leaders These bar charts isolate the top performers from the optimization frontier.

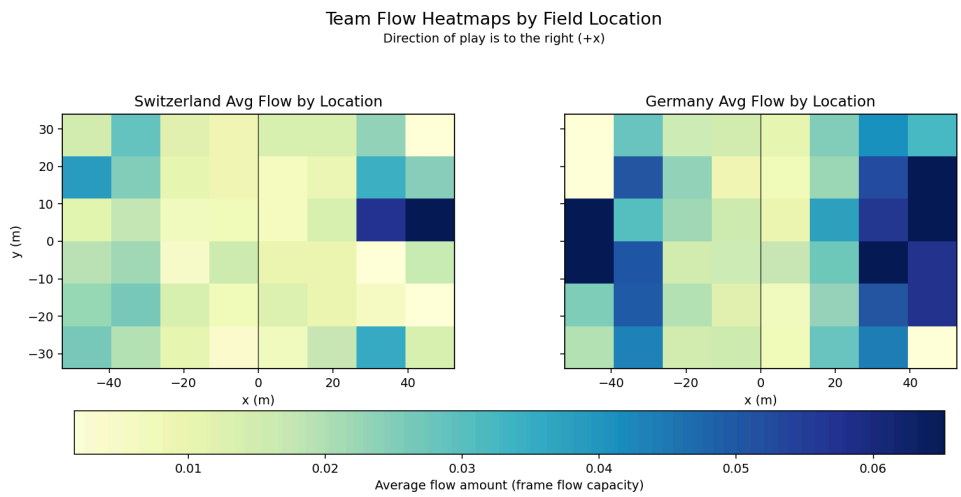
- **Highest Optimal Action Rate (Left):** Ranks the players who most frequently chose the highest-value action available to them, with E. Cömert and F. Wirtz leading the metric at nearly 70%.
- **Lowest Mean Regret (Right):** Highlights players who minimize offensive value loss. Even when these players do not choose the strict mathematical optimum, their alternative choices still maintain high network flow, resulting in the lowest average regret.



This heatmap illustrates where teams lose potential offensive value due to suboptimal decision-making.

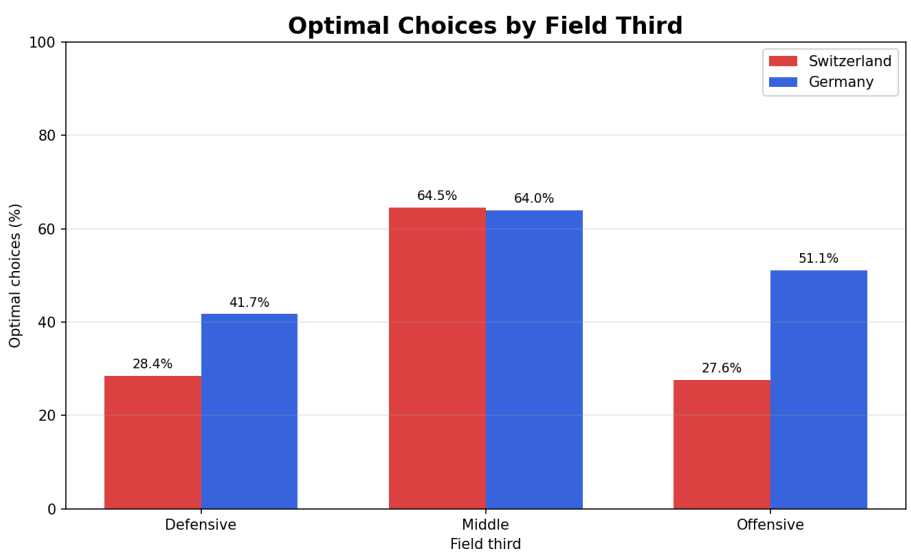
- **Switzerland (Left):** Highest regret occurred along the offensive sidelines, indicating struggles to maximize value from wide areas via crosses or passes.

- **Germany (Right):** Regret was heavily concentrated in the congested central offensive areas (Zone 14), highlighting the difficulty of executing the mathematically optimal action in tight, final-third spaces.



This visualization maps the average network flow generated when a team possesses the ball in specific zones, translating our metric into spatial dominance.

- **Germany's Bimodal Threat:** Germany generated peak flow during deep build-up (near their own keeper) and deep in the final third, indicating both secure progression and dangerous attacking structures.
- **High Press Impact:** Comparing the defensive thirds highlights the effectiveness of Germany's high press. They severely restricted Switzerland's flow near their own goal, successfully cutting off passing lanes and stifling their build-up play.



This bar plot breaks down the percentage of mathematically optimal decisions made by each team across the defensive, middle, and attacking thirds.

- **Attacking Third Disparity:** The most significant difference occurred in the offensive third, where Germany executed the optimal action 50% of the time compared to Switzerland's 27%. This highlights Germany's superior composure and ability to find the highest-value option in critical scoring areas.
- **Middle and Defensive Thirds:** Both teams performed efficiently in the middle third with a high and evenly matched rate of optimal choices. However, Germany maintained a slight edge in the defensive third, indicating more secure and calculated build-up play under pressure.

Continued in Appendix: Technical Implementation Details →

APPENDIX

Technical Implementation Details

Complete methodology, code references, and data schemas

A1. Ball Carrier Identification

Before building the network, the model must know where the flow originates. Ball carrier resolution uses a two-tier hybrid system:

5. **Primary — SkillCorner AI Detection:** If the tracking frame includes a possession field with a player_id, that player is immediately used. This is the fastest and most reliable path.
6. **Fallback — Closest Player with Distance Gate:** When AI detection is unavailable, the model queries a pre-built NumPy integer array (frame_team_array) for O(1) possession lookup, then finds the closest player from the possessing team to the ball. A hard maximum threshold of 2.0 meters is enforced — if no player is within range, the ball is considered loose and the frame returns None.

The phases-of-play array is pre-built at startup for O(1) per-frame performance:

```
# Vectorized NumPy slice assignment for O(1) lookup
frame_team_array = np.full(max_frame + 1, -1, dtype=np.int32)
for _, row in df_phases.iterrows():
    frame_team_array[row.frame_start : row.frame_end + 1] = team_id
```

This avoids expensive DataFrame filtering on every frame and reduces per-frame possession lookup to a single array index operation.

A2. Tracking-Aware Expected Goals — Full Detail

Layer 1: Distance and Angle (Base Probability)

The baseline xG is computed using the Euclidean distance to the goal center and the visible arc between the goalposts, mapped through a logistic sigmoid:

> $$base_xg = 1 / (1 + \exp(-1.0 - 0.15 \cdot d + 1.5 \cdot \theta))$$

d = distance to goal center in meters. θ = visible angle between the two goalposts in radians, computed using the arctangent formula with the standard 7.32m goalpost width.

Layer 2: Defenders in the Shot Cone

A triangle is drawn from the shooter's position to both goalposts. All opponents physically inside this cone apply a compounding 30% penalty:

> $$adjusted_xg = base_xg \times (0.70)^{n_defenders_in_cone}$$

Two defenders in the cone: ~51% reduction. Three defenders: ~66% reduction. Each defender is checked via a standard point-in-triangle test.

Layer 3: Defensive Pressure

If any opponent is within 1.5 meters of the ball carrier at shot evaluation time, a further 40% penalty is applied:

> ***if nearest_opponent ≤ 1.5m: xg ×= 0.60***

Layer 4: The 45m Hard Cap

Shots from beyond 45 meters are assigned xG = 0.0. This prevents the logistic formula from generating positive probabilities for statistically impossible long-range attempts in open play.

Coordinate Normalization

The xG formula assumes the attacking team always shoots toward +x. Raw SkillCorner coordinates must be normalized using period-and-team metadata before any calculation:

```
def get_team_attack_sign(team_id, period, match_data):
    home_dir = match_data['home_team_side'][period - 1]
    if team_id == home_team_id:
        return +1 if home_dir == 'left_to_right' else -1
    else:  # away team
        return -1 if home_dir == 'left_to_right' else +1

norm_x, norm_y = sign * raw_x, sign * raw_y
```

A3. Pass Utility and xThreat Economics

Pass edges are weighted by a full economic utility formula rather than raw completion probability:

> ***pass_utility = (xPass × xThreat) − ((1 − xPass) × xCost)***

Variable	Source	Physical Meaning
xPass	SkillCorner AI (player_targeted_xpass_completion)	Probability of the pass completing given current defensive positions
xThreat	SkillCorner AI (player_targeted_xthreat)	Expected goal probability from the receiver's current pitch position
xCost	Karun Singh 12×8 xT grid (scipy interpolated)	Expected threat for the opponent if they intercept at the receiver's location

The xCost term is interpolated from Karun Singh's open-source 12x8 Expected Threat grid. For the opponent's perspective, coordinates are mirrored before interpolation to ensure correct grid orientation regardless of attacking direction.

A4. Run Option Modeling — Full Penalty System

13 directional run destinations are evaluated per frame: 8 forward-facing at −95° to +95° around the goal-direction, 4 backward-facing, plus the carrier's current velocity direction. Each destination is ~7m from the carrier's position.

Zone Control

For each destination, a movement-aware zone control score is computed using player arrival times:

```
t_att = min(arrival_time for attacker in attackers at destination)
t_def = min(arrival_time for defender in defenders at destination)
delta_t = clip(t_def - t_att, -8.0, 8.0)
zone_control = 1 / (1 + exp(-1.3 * delta_t))   # Logistic, beta=1.3
```

Arrival time for each player accounts for current velocity, maximum speed, acceleration, and reaction time. The zone control value becomes the capacity of the source-to-run-node edge.

Run Net xThreat Penalty Components

Base Pressure	Defender proximity and forward-directional weighting near the destination.
Lane Interception	Defenders inside the 2.25m-wide running corridor. High density generates higher penalties.
Directional Crowding	Defenders within 12m ahead and 3.5m laterally in the run direction.
Front Blocker	Defenders directly ahead who would physically obstruct the run. Penalty scales with proximity to goal (box_proximity) and count.
Local Congestion	Exponentially scaling penalty per additional nearby defender. Each extra nearby defender multiplies penalty by ~1.8×.
Near-Ball Pressure	When the carrier themselves is under immediate pressure, all run values are discounted.
Non-Progress	Flat penalty applied when the run does not make positive progress toward goal.

The run capacity entering the network is min(zone_control, run_net_xthreat). Only the top-5 positive-value runs are added to the graph to prevent directional over-representation.

A5. Network Construction and Ford-Fulkerson Execution

With all components computed, the capacity graph is assembled and the maximum flow is calculated:

```
capacity_graph = {
 'SOURCE_42': {
   'PLAYER_7':  0.214,  # pass utility to player 7
   'PLAYER_11': 0.089,  # pass utility to player 11
   'RUN_forward_2': 0.063,  # zone control capacity
   'RUN_forward_5': 0.041,
   'GOAL': 0.031,       # direct xG shot edge
```

```
 },
 'PLAYER_7':      { 'GOAL': 1.0 },
 'PLAYER_11':     { 'GOAL': 1.0 },
 'RUN_forward_2': { 'GOAL': 0.058 },
 'RUN_forward_5': { 'GOAL': 0.039 },
 'GOAL': {}
}

max_flow, flow_dict, residual = ford_fulkerson_max_flow(
   capacity_graph, source='SOURCE_42', sink='GOAL'
)
```

A6. Full-Game Precomputation and Data Export

The model's precompute mode processes every tracking frame in the dataset and exports a structured CSV cache, decoupling heavy computation from downstream visualization.

Column	Type	Description
frame	int	Frame number in the tracking sequence
timestamp / period	string/int	Match timestamp and period (1 or 2)
team_side	string	'home', 'away', or 'none' based on current possession
carrier_id / carrier_x / carrier_y	int/float	Ball carrier ID and pitch coordinates
frame_flow_capacity	float	Maximum flow scalar for this frame (absolute value)
flow_proxy	float	Signed flow: +ve for home team, −ve for away team
best_pass_utility	float	Highest pass utility among all available options
best_run_value	float	Highest run net xThreat (positive values only)
shot_xg	float	Direct xG from current ball position
best_action	string	'pass' or 'shot' based on which has higher value
pass_options_json	JSON	Full per-receiver pass data: xpass, xthreat, xcost, utility
run_options_json	JSON	All evaluated run directions with run_value per destination
capacity_graph_json	JSON	Complete frame-level network capacity graph (all edges)
flow_dict_json	JSON	Actual routed flow on each edge after Ford-Fulkerson

Signed Flow Convention

- **Home team in possession:** flow_proxy = +frame_flow_capacity (above zero in chart)
- **Away team in possession:** flow_proxy = −frame_flow_capacity (below zero in chart)
- **No clear possession:** flow_proxy = 0.0

The live animation renders this as a split chart with the home team filling upward and the away team filling downward. Gaps in flow are forward-filled via pandas ffill() to prevent visual jumps.

A7. Development Challenges and Solutions

Challenge 1: The Goalie Identity Illusion

Early iterations assigned enormous xG values to goalkeepers and defenders near their own goal during build-up play — because the logistic formula correctly rewarded proximity to goal, but the model did not know which goal was being attacked.

Solution: Period-Aware Attacking Direction Normalization

All coordinates are multiplied by a sign factor (+1 or −1) derived from match metadata before any xG or xThreat calculation, ensuring the formula always sees the ball moving toward +x regardless of which half of the pitch is being attacked in the current period.

Challenge 2: xCost Grid Orientation

The Karun Singh xT grid assumes left-to-right attack. When computing xCost from the opponent's perspective, a second mirroring step was required:

```
if sign == +1:   # We attack right, opponent attacks left
    opp_x = 105.0 - spadl_x
    opp_y = 68.0  - spadl_y
else:            # We attack left, opponent attacks right
    opp_x = spadl_x
    opp_y = 68.0  - spadl_y
```

Challenge 3: Performance at Scale

Processing ~54,000 frames for a 90-minute match at 10fps requires per-frame zone control calculations, 13-direction run sweeps, and Ford-Fulkerson execution. Key optimizations:

- **O(1) phases array:** Vectorized NumPy slice assignment replaces per-frame DataFrame filtering.
- **Pre-compiled xT interpolator:** scipy RegularGridInterpolator with boundary extrapolation avoids repeated array searching.
- **Run option capping:** Only the top-5 positive-value run options enter the flow graph.
- **JSON-safe serialization:** Recursive converter handles NumPy types, NaN, and infinite floats before CSV export.